

Project Summary

The project titled “**Introduction to Amazon DynamoDB**” was carried out using Amazon Web Services (AWS) to understand the working of cloud-based NoSQL database systems. The main objective of the project was to learn how to create, manage, query, and delete data in Amazon DynamoDB using the AWS Management Console. DynamoDB is a fully managed NoSQL database service provided by [Amazon Web Services](#) that offers high scalability, low latency, and flexible data storage for modern applications.

In this project, a music library database was created to store information related to songs, artists, albums, and other related details. The project demonstrated how DynamoDB handles data without requiring a fixed schema, which makes it more flexible compared to traditional relational databases. Different database operations such as creating a table, adding records, updating existing records, querying data, scanning records using filters, and deleting the table were successfully performed. Through this project, practical knowledge about NoSQL databases and cloud database management was gained.

Problem Statement

Modern applications such as mobile applications, web platforms, gaming systems, and IoT services require databases that can handle large volumes of data efficiently while maintaining high performance and scalability. Traditional relational database systems often require predefined schemas and may face limitations when handling rapidly changing or semi-structured data.

The problem addressed in this project was to understand how Amazon DynamoDB can be used as a flexible and scalable NoSQL database solution for storing and managing data efficiently in a cloud environment. The project aimed to implement basic database operations using DynamoDB and analyze how NoSQL databases provide advantages such as schema flexibility, faster data retrieval, automatic scaling, and serverless management.

The project specifically focused on creating a music library database where different music records could be stored, modified, queried, and deleted using DynamoDB services provided by AWS.

Architecture

The architecture of the project was designed using AWS cloud services and Amazon DynamoDB as the core database system. The implementation was performed through the AWS Management Console, which provided a graphical interface to create and manage database resources.

Amazon DynamoDB was used as the primary NoSQL database service for storing and managing music-related data. DynamoDB is a fully managed cloud database service that supports key-value and document-based data models. It automatically handles database management tasks such as scaling, performance optimization, storage management, and availability.

A table named **Music** was created in DynamoDB to store information about songs and artists. The table was configured using a primary key structure consisting of:

- Partition Key: Artist (String)
- Sort Key: Song (String)

The partition key was used to uniquely identify and distribute records across DynamoDB servers, while the sort key helped organize songs belonging to the same artist and enabled efficient querying.

Each record in the table was stored as an item containing multiple attributes such as Artist, Song, Album, Year, Genre, and LengthSeconds. Different items contained different attributes, demonstrating the schema-less nature of DynamoDB.

The architecture also included query and scan operations for retrieving data. Query operations were performed using partition and sort keys for faster access, while scan operations were used with filters to search through the complete table data.

Overall, the project followed a serverless cloud architecture where AWS managed the complete backend infrastructure, database scaling, and resource availability automatically.

Steps of Implementation

Step 1: Launching the AWS Environment

The AWS Skill Builder lab environment was started, and the AWS Management Console was opened. The DynamoDB service was accessed from the AWS console dashboard.

Step 2: Creating the DynamoDB Table

A new table named **Music** was created in DynamoDB. The following configurations were provided:

- Table Name: Music
- Partition Key: Artist (String)
- Sort Key: Song (String)

The screenshot shows the 'Create table' wizard in the AWS Management Console. It is divided into three main sections: 'Table details', 'Partition key', and 'Table settings'. In the 'Table details' section, the table name 'Music' is entered. The 'Partition key' section shows 'Artist' as the partition key with a 'String' data type. The 'Sort key - optional' section shows 'Song' as the sort key with a 'String' data type. In the 'Table settings' section, the 'Default settings' radio button is selected, indicating that the table will be created with the fastest way to create a table.

Create table

Table details [info](#)
DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name
This will be used to identify your table.
Music
Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).

Partition key
The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.
Artist String
1 to 255 characters and case sensitive.

Sort key - optional
You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.
Song String
1 to 255 characters and case sensitive.

Table settings

☒ **Default settings**
The fastest way to create your table. You can modify most of these settings after your table has been created. To modify these settings now, choose 'Customize settings'.

☐ **Customize settings**
Use these advanced features to make DynamoDB work better for your needs.

Default table settings were selected, and the table was created successfully.

Step 3: Adding Data into the Table

Multiple music records were inserted into the Music table using the “Create Item” option.

The first record stored information about the song *Money* by Pink Floyd with attributes such as Album and Year.

The second record stored details about *Imagine* by John Lennon and included an additional attribute called Genre.

The third record stored details about *Gangnam Style* by Psy and included another attribute called LengthSeconds.

This step demonstrated DynamoDB's flexibility because different records were allowed to contain different attributes without requiring a fixed schema.

Attribute name	Value	Type	Remove
Artist - Partition key	Pink Floyd	String	
Song - Sort key	Money	String	
Album	The Dark Side of the Moon	String	Remove
Year	1973	Number	Remove

Step 4: Modifying an Existing Item

An existing item in the table was modified using the “Edit Item” option. The record for the song *Gangnam Style* by Psy was updated by changing the Year value from 2011 to 2012. The updated data was then saved successfully.

Step 5: Querying the Table

The Query operation was performed to retrieve a specific record from the Music table. The query was executed using:

- Partition Key: Artist = Psy
- Sort Key: Song = Gangnam Style

Attribute	Value
Artist	Psy

Attribute	Value
Song	Gangnam Style

Filters - optional

Run Reset

Completed - Items returned: 1 - Items scanned: 1 - Efficiency: 100% - RCUs consumed: 0.5

Table: Music - Items returned (1)

Query started on May 10, 2026, 13:14:52

Artist (String)	Song (String)	Album	LengthSeconds	Year
Psy	Gangnam Style	Psy 6 (Six R...	219	2012

The required record was retrieved successfully using indexed key values.

Step 6: Scanning the Table

A Scan operation was performed to search records using filters. A filter condition was applied where the Year attribute was equal to 1971. The operation returned the record for the song *Imagine* by John Lennon.

The screenshot shows the AWS DynamoDB console interface. On the left, the 'DynamoDB' sidebar is visible with options like Dashboard, Tables, Explore items, etc. The main panel shows the 'Music' table selected. The 'Scan or query items' section is active, with 'Scan' selected. The 'Select a table or index' dropdown shows 'Table - Music'. The 'Select attribute projection' dropdown shows 'All attributes'. Under 'Filters - optional', a filter is added: 'Attribute name: Q Year', 'Condition: Equal to', 'Type: Number', 'Value: 1971'. The 'Run' button is highlighted. Below the filter section, a green status bar indicates 'Completed - Items returned: 1 - Items scanned: 3 - Efficiency: 33.33% - RCUs consumed: 2'. The 'Table: Music - Items returned (1)' section shows a single record: 'John Lennon' with 'Song (String): Imagine', 'Album: Imagine', 'Genre: Soft rock', and 'Year: 1971'.

Step 7: Deleting the Table

After completing all operations, the Music table was deleted successfully from DynamoDB. The deletion request was confirmed, and all stored records were removed from the database.

The screenshot shows the AWS DynamoDB console with the 'Delete table' dialog box open. The dialog asks: 'Delete table Music in Asia Pacific (Sydney) permanently? This action cannot be undone.' It includes a warning: 'Proceeding with this action will delete the table and you won't be able to retrieve this data.' There are two checkboxes: 'Delete all CloudWatch alarms for Music.' (checked) and 'Create an on-demand backup of Music before deletion.' (unchecked). A text input field contains 'confirm'. The 'Delete' button is highlighted.

Output

The project was implemented successfully using Amazon DynamoDB in the AWS cloud environment. A Music table was created with appropriate partition and sort keys, and multiple records were inserted successfully into the database. The update operation modified existing data correctly, while query and scan operations retrieved the expected records efficiently.

The project successfully demonstrated the working of a NoSQL cloud database system and provided practical understanding of database creation, data insertion, querying, scanning, updating, and table deletion in Amazon DynamoDB.

